Activity 5 Report: Database Maintenance and Monitoring

COMP 018 Database Administration | Project tested: CampusSafe (Campus Hazard and Safety Reporting System) (PHP + PostgreSQL) | Date of testing: 7 November 2026

1. Cover and links

Item	Value
Testing group	G08: Borreta, Joshua; Bola, Martin Lawrence; Juatchon, Joaquin Phillip; Marcos, Karl Raphael; Pacheco Jr., Sancho; Vera Cruz, Euan Joseph
Project tested	CampusSafe (Campus Hazard and Safety Reporting System)
Owning group	Group 7 (member names are not listed in the project files)
Repository	grp07-campussafe (project zip grp07-campussafe-main; the repository link is not included in the files we received)
Tracker	https://github.com/EclipseChan/bwisIT-Solutions---Campus-Canteen-Pre-Order-and-Queue-Management-System/tree/main/other_group_tests
Evidence files	a5_maintenance.txt

Every result below comes from this run on a freshly created database.

Run at about 00:58 on 7 November 2026 (Asia/Manila), after the application and access tests.

2. Health and activity

Check	Result
Version	PostgreSQL 18.6, x86_64 Windows
Database size	9,174 kB
Connections	1 backend (the monitoring session). max_connections is 100
Transactions	1,431 commits, 27 rollbacks
Cache	188,184 buffer hits against 48 reads, about 99.97% hit ratio
Deadlocks and temp files	0 and 0
Autovacuum	on. It had not yet processed these tables, since the data is small (last_autovacuum is empty for all tables)
Rows	role 5, app_user 7, category 8, location 10, hazard_report 19, assignment 13, history 68, actions 11, attachment 5
Largest tables	hazard_report 112 kB, assignment 72 kB, app_user 64 kB; the rest 32 to 48 kB

Dead tuples before maintenance: app_user 7, hazard_report 6, assignment 2, all other tables 0. These come from updates during the tests (verification and assignment, the role statement in Activity 4, password reseeding).

3. Index usage

24 indexes. Heavily used: app_user_pkey (224 scans), hazard_report_pkey (120), campus_location and hazard_category primary keys (72 each), role_pkey (39) and role_role_name_key (30), idx_report_status (14), idx_assign_report (11). Never used so far: idx_report_submitted, idx_report_category, idx_report_location, app_user_email_key, hazard_category_category_name_key, maintenance_action_pkey and report_status_history_pkey. The unique and primary key indexes exist to enforce rules, so being unused is normal for them. idx_report_submitted, idx_report_category and idx_report_location have simply not been needed on 19 rows.

Foreign key columns with no index: app_user.role_id, hazard_report.verified_by_user_id, assignment.assigned_by_user_id, report_status_history.changed_by_user_id, maintenance_action.maintenance_user_id, attachment.uploaded_by_user_id.

4. Logging

Setting	Value	Comment
logging_collector	on	Server log written to the log folder
log_statement	none	No statement logging
log_min_duration_statement	-1	Slow queries are not logged
Application log	PHP console only	The app has no log file of its own and shows some database errors on the page

5. Log findings

The server log is shared by everything on this server. The entries for this project start at 00:54 on 7 November.

Time	Entry	Explanation
00:54:29	violates check constraint "hazard_report_status_check", "chk_resolved_has_date", "hazard_report_priority_check"	Our three direct database tests in Activity 3
00:57:55 to 00:57:57	permission denied for table hazard_report, app_user, hazard_category, attachment; vacuum skipped with a warning	The real-login tests of Activity 4 (report and backup accounts, and the denied delete and truncate as the app account)

Every entry has a cause in the test steps. Application-level validation errors (empty description, bad login) never reach the database log because the PHP code rejects them first.

6. Maintenance performed

Action	Result
ANALYZE (whole database)	Completed. last_analyze set on every table
VACUUM (VERBOSE, ANALYZE) hazard_report	Completed. Dead tuples 6 to 0
VACUUM (VERBOSE, ANALYZE) report_status_history	Completed. 68 live rows, 0 dead
Dead tuples afterwards	hazard_report 0, report_status_history 0, assignment 2 and app_user 7 (not vacuumed manually)
VACUUM and ANALYZE as campussafe_app	Skipped: permission denied (not the table owner). The project defines no maintenance role

7. Query plans

Five queries the pages use were run with EXPLAIN (ANALYZE, BUFFERS): the report list with its joins, the verification queue, one report's history, a technician's assignments, and the status count for the dashboard and statistics.

Query	Plan	Planning / execution
Report list (join with category and location)	Sequential scans with hash joins	5.13 ms / 0.12 ms
Verification queue (status = Submitted, order by date)	Sequential scan and sort	3.00 ms / 0.04 ms
History of one report	Sequential scan	1.48 ms / 0.04 ms
A technician's assignments (join with app_user)	Sequential scans	3.83 ms / 0.05 ms
Count by status	Sequential scan and aggregate	2.96 ms / 0.07 ms

With 19 reports every query is instant and a sequential scan is the right plan, so the unused indexes cannot be judged yet. Planning takes longer than execution in all five cases, which is normal for tiny tables. The plans should be checked again with a few thousand reports.

8. Recommendations

- Create a maintenance role (member of pg_maintain) for scheduled VACUUM and ANALYZE, because campussafe_app cannot run them.
- Set log_min_duration_statement to about 500 ms and log_statement to ddl.
- Add indexes on the foreign key columns listed in section 3 when the tables grow, starting with maintenance_action.maintenance_user_id and hazard_report.verified_by_user_id.
- Check idx_report_submitted, idx_report_category and idx_report_location again after real use and drop them if they stay unused.
- Give the PHP side a log file and log the real database error instead of showing it on the page.

9. Feedback for the owners

No health problems found. The suggestions are a maintenance role, slow-query logging, and a review of the unused indexes once there is real data.

AI usage declaration

- Tool: Claude (Anthropic).
- Tasks: interpreting the results of the testing.
- Validation: the commands were run by the group on its own computer, and every result in this report comes from the saved output files.